Bottleneck Fixes — Benchmark Validation

AI Civilization Engine · findings, shipped fixes, and reproducible A/B micro-benchmarks

Generated 2026-07-17 · branch `claude/project-bottlenecks-rww3q8` · fleet scope: 20 villagers

PART 1 — BENCHMARK VALIDATION

Bottleneck-fix benchmark report

Faithful in-process A/B micro-benchmarks for the shipped bottleneck fixes in agent-service and memory-service. Each bench runs the SAME workload through the shipped code path (treatment) and a reconstruction of the pre-fix path (baseline), discards a warm-up window, and reports p50/p95 over the remaining iterations. Backends (GPU, broker, DB, embedding endpoint) are modeled deterministically so numbers are reproducible and reflect the structural change, not host noise or a live GPU. The event-service (Java) and minecraft-service (Node) fixes ship with their own in-language test suites and are out of scope for this Python in-process harness — see the bottleneck report for those. Run: `cd services/agent-service && uv run python ../../bench/run_all.py`.

Headline

- **LLM concurrency gate (agent-service #1, Critical)** — `batch_wall_ms` -67.6% (p50)
- **Kafka producer batching (agent-service #5)** — `awaited_round_trips` -87.5% (p50) · behaviour preserved
- **Batched relationship upserts (agent-service #7)** — `transactions` -83.3% (p50) · behaviour preserved
- **Query-embedding cache (memory-service, Tier-3)** — `embedding_round_trips` -75.0% (p50) · behaviour preserved

Method notes

- **Warm-up discarded** every bench drops its first iterations before measuring — the project's own MSPT rule (post-boot world-gen spike is not steady state).
- **Real code under test** the LLM gate drives the shipped `OllamaProvider` semaphore; the producer bench drives the shipped `EventPublisher.publish` / `flush`; the relationship correctness fold uses the shipped `_clamp` and `GRUDGE_AFFINITY_THRESHOLD`; the query-cache bench drives the shipped `QueryEmbeddingCache` LRU over a real `FakeEmbeddingProvider`.
- **Models, not live infra** GPU/broker/DB/embedding latencies are modeled so the delta isolates the structural change. Whole-stack soak numbers (Prometheus before/after) are the next layer and need a dedicated load run.

Benches

LLM concurrency gate (agent-service #1, Critical)

20 concurrent ticks against one modeled GPU that thrashes past 4 in-flight. Baseline = ungated fan-out; treatment = shipped semaphore cap of 4.

Confirms: bottleneck-report §1 / providers.py:194 (asyncio.Semaphore backpressure)

metric	baseline p50	baseline p95	treatment p50	treatment p95	Δ p50
`batch_wall_ms` ←	245	263	79.3	103	-67.6%
`call_max_ms`	241	255	25.7	26.3	-89.3%
`call_p50_ms`	95.5	103	16.5	18.0	-82.7%

Kafka producer batching (agent-service #5)

8 events/tick. Baseline = send_and_wait per event; treatment = real EventPublisher (buffered send() + one flush()).

Confirms: bottleneck-report §5 / producer.py:26,33 (fire-and-forget + one flush)

Behaviour-preservation: ✅ PASS — sends=8 (want 8), flushes=1 (want 1), round_trips=1 (want 1)

metric	baseline p50	baseline p95	treatment p50	treatment p95	Δ p50
`awaited_round_trips` ←	8.00	8.00	1.00	1.00	-87.5%
`modeled_tick_ms`	16.0	16.0	2.00	2.00	-87.5%

Batched relationship upserts (agent-service #7)

6 edges moved in one tick. Baseline = one transaction per edge; treatment = a single batched transaction. Correctness folds 4000 random sequences both ways and asserts identical final edges.

Confirms: bottleneck-report §7 / relationships.py:132 (one session per tick's batch)

Behaviour-preservation: ✅ PASS — 4000 random update sequences: batched == sequential (bounds + grudge damping preserved)

metric	baseline p50	baseline p95	treatment p50	treatment p95	Δ p50
`db_round_trips`	12.0	12.0	3.00	3.00	-75.0%
`modeled_tick_ms`	18.0	18.0	4.50	4.50	-75.0%
`transactions` ←	6.00	6.00	1.00	1.00	-83.3%

Query-embedding cache (memory-service, Tier-3)

20 retrievals in one window over 5 distinct salient queries. Baseline = embed every read; treatment = shipped QueryEmbeddingCache (one backend round-trip per distinct key).

Confirms: bottleneck-report Tier-3 / embeddings.py:29,53 (LRU on the read path)

Behaviour-preservation: ✅ PASS — 20 reads over 5 distinct keys: backend hit 5x (want 5); all cached vectors == uncached truth

metric	baseline p50	baseline p95	treatment p50	treatment p95	Δ p50
`embedding_round_trips` ←	20.0	20.0	5.00	5.00	-75.0%
`modeled_window_ms`	160	160	40.0	40.0	-75.0%

Raw data

Per-iteration CSV and JSON summaries are in `bench/results/*.csv` / `*.json`.

PART 2 — FULL BOTTLENECK REPORT

AI Civilization Engine — Bottleneck Report

Date: 2026-07-17 · **Scope:** whole system at the current 20-villager fleet · **Method:** parallel code sweeps of agent-service, memory-service, event-service, government-service, minecraft-service, and the infrastructure/dashboard layer, cross-checked against the July 2026 CPU profiling pass (PR #34).

Executive summary

The system's remaining bottlenecks are almost all **structural serialization points**: places where work that could run in parallel funnels through one thread, one lock, one partition, or one GPU. The concurrency fundamentals are sound — villager ticks are genuinely async and staggered, the ledger reads use keyset pagination, the SSE feed is push-based, and the PR #34 pathfinder/CPU fixes are holding. What remains is a set of choke points that are cheap to widen now and expensive to hit later.

#	Bottleneck	Where	Severity
1	Single shared LLM backend serializes all deliberation	agent-service → Ollama/OpenAI	**Critical**
2	Ledger ingest single-threaded; SSE fan-out runs on the same thread	event-service	**High**
3	Cross-villager head-of-line blocking on the command topic	minecraft-service / Kafka	**High**
4	pgvector retrieval degrades as memory streams grow (no pruning)	memory-service	**High (creeping)**
5	No Kafka producer batching anywhere (per-event awaited sends)	agent- & minecraft-service	Medium
6	Vote casting lock-serialized per election	government-service	Medium
7	Per-update relationship transactions on a 5-connection pool	agent-service	Medium
8	One untuned stock Postgres hosts all five databases; no container limits	infrastructure	Medium
9	Silent single-partition regression path (topic auto-create still on)	Redpanda	Medium (latent)

Tier 1 — the walls you hit first

1. The LLM backend is the hard throughput ceiling

All 20 tick loops run concurrently, but each one awaits `provider.complete()` against a **single shared backend** — one Ollama instance on one GPU, which serializes token generation. The scheduler's own docstring admits "91% worst-case duty on the 4090" (`agent_service/brain/scheduler.py:7`). Tick staggering (`scheduler.py:87`) only hides the problem while `N × avg_deliberation_seconds < TICK_INTERVAL_SECONDS`; reactive ticks bypass the stagger and can stampede the GPU.

This is *the* limit on fleet size. Nothing else in the codebase matters for scaling until this does.

- **Cheapest mitigation:** a concurrency semaphore around `complete()` so overflow queues instead of thrashing VRAM; tune `TICK_INTERVAL_SECONDS` against fleet size.
- **Real fix:** multiple LLM workers or a hosted endpoint with true concurrency.

2. event-service ingests the entire system single-threaded — and slow browsers can throttle it

One `@KafkaListener` with default concurrency = 1 consumes **all six topics** (30 partitions total) — `EventEnvelopeConsumer.java:36`, with no `ConcurrentKafkaListenerContainerFactory` anywhere. Each record is one `INSERT` round-trip plus one per-record offset ack (`JdbcEventStore.java:33-56`, `ack-mode: record`). No JDBC batching, no batch listener.

Worse, SSE fan-out runs **inline on that same consumer thread**: `EventIngestService.java:41` calls `live.publish(event)`, and `SseBroadcaster.java:59-79` loops every connected emitter doing a blocking `send()`. A handful of slow dashboard clients directly back-pressure Kafka ingest for the whole system — the read path throttles the write path.

- **Fixes (all cheap, high leverage):** set listener `concurrency ≥ 3` (partitions already exist); switch to a batch listener with multi-row inserts or `reWriteBatchedInserts=true`; hand SSE fan-out to an async executor.
- Also missing: an index on `aggregate_type` (a supported REST filter falls back to a full ledger scan — `JdbcEventStore.java:67-70`), and `SELECT *` ships the unused stored `tsvector` on every paged read (`JdbcEventStore.java:60`).

3. Cross-villager head-of-line blocking on the command topic

The single-partition hypothesis is dead — `commands.minecraft` has 6 partitions and the consumer sets `partitionsConsumedConcurrently: 6` (`commandConsumer.ts:49`). But 20 bots hash into 6 partitions (~3.3 bots each), and `eachMessage` **awaits the action to completion** (`executor.ts:232`): the watchdog `Promise.race` resolves only when the action finishes or times out. A bot on a 30-second gather freezes its 2–3 partition-mates for the whole action, even though per-villager ordering only requires same-key serialization.

Amplifier: `payload.timeoutMs` is used **unclamped** in the watchdog `setTimeout` (`executor.ts:186-197`) — the `TIMEOUT_TABLE_MAX_MS` cap lives in the caller contract, not the executor. One oversized value pins a partition and its neighbors for the duration.

- **Fixes:** raise the partition count toward the bot count (requires the documented drain→recreate migration — `docs/runbooks/kafka-topic-migration.md`); clamp with `Math.min(payload.timeoutMs, MAX_MS)` in the executor.

4. pgvector retrieval degrades as memory streams grow

The retrieval query filters `WHERE villager_id = :id` and orders by cosine distance (`memory_service/service.py:143-152`) — but the HNSW index cannot use the villager predicate. It walks the **global** vector graph at the default `ef_search = 40` and post-filters by villager. As total memory count grows, per-villager recall drops and latency climbs. Three compounding factors:

- **No pruning, decay, or consolidation anywhere** — every villager's stream grows forever, and reflections add rows.

- The reflection scheduler runs a **full-table** `SUM(importance) … GROUP BY villager_id` every 300 s (`reflection.py:96-112`) with no supporting index on `created_at` — cost scales with total table size.
- The architecture docs defer this to "100+ villagers" (`02-database.md:391`), but the driver is *total row count over time*, not villager count — a long-running 20-villager world hits it too.
- **Fixes:** `SET LOCAL hnsw.ef_search` per query (sized to `k × candidate_factor × 4`); index `(villager_id, memory_type, created_at)` for the reflection scan; a periodic decay/archival job for low-importance, long-untouched observations.

Tier 2 — real costs, not yet walls

5. No Kafka producer batching anywhere

- **agent-service:** 4–8 serial `send_and_wait` round-trips per tick (`producer.py:26`) — DecisionMade, ActionRequested, one RelationshipChanged per update, VillagerTalked, MemoryFormed — each an awaited broker round-trip with no `linger_ms`. **Fix:** fire-and-forget `send()` per event, one `flush()` per tick.
- **minecraft-service:** producer configured with kafkajs defaults (`producer.ts:11`) — `acks=-1`, `lingerMs=0`, so every world event is an un-batched full-ISR-ack request. The awaited `publishOutcome` (`executor.ts:176`) adds its round-trip to the head-of-line problem in §3; `gather` emits one produce request per block collected (`BotSession.ts:877-885`). **Fix:** `lingerMs: 5-20`, consider `acks: 1` for the world-event stream.

6. Vote casting is lock-serialized

Every ballot takes `FOR UPDATE` on the election row, then runs 3–4 statements — a `findVote` pre-check, a full `candidatesOf` list load to validate one candidate, then the insert (`ElectionService.java:158-198`, `JdbcElectionStore.java:77-158`). All concurrent voters for a hot election process strictly serially regardless of thread count. The `votes_one_per_voter` UNIQUE constraint already makes the insert idempotent, so the pessimistic lock isn't needed for the common path.

- **Fixes:** drop the row lock from the vote path (keep it for the election clock's state transitions); replace the candidate-list load with a `SELECT 1 … WHERE election_id=? AND id=?` existence check. Related: `latest()` is a 1+3N query pattern (`ElectionService.java:132-140`) — batch when the archive grows.

7. Per-update relationship transactions on a small pool

Each relationship change opens its own session, does `SELECT … FOR UPDATE`, commits, then publishes — N sessions, N round-trips, N publishes per tick (`graph.py:268-306`, `relationships.py:102-144`). The engine pool is `pool_size=5` (default `max_overflow=10`) shared by 20 concurrent tick loops (`db.py:9`); memory-service has the same sizing (`memory_service/db.py:7`). When chatty ticks cluster, checkouts block.

- **Fixes:** batch the edge upserts in one transaction (`INSERT … ON CONFLICT DO UPDATE` over the update list); collect RelationshipChanged envelopes for one flush; raise `pool_size` / `max_overflow`.

8. One untuned Postgres hosts all five databases; nothing has resource limits

A single `pgvector/pgvector:0.8.0-pg16` container hosts agent_db, memory_db, event_db, government_db, and analytics_db with **zero tuning** — stock `shared_buffers=128MB` — while HNSW vector search, ledger appends, and relationship writes fight for the same page cache and I/O. And **no compose service has any resource limit**: the 3 G Minecraft JVM, Redpanda, Postgres, and every app service run unbounded on the host, so any one of them can starve the others.

- **Fixes:** mount a tuned `postgresql.conf` (`shared_buffers` , `work_mem`); add `mem_limit` to at least postgres, redpanda, and minecraft.

9. Silent single-partition regression path

Redpanda topic auto-create is still on. A producer that beats `scripts/provision-topics.mjs` on a fresh cluster silently recreates its topic at **1 partition** — the exact bug that made decision→speech gaps "run minutes" before M2-4 (`docs/architecture/08-m2-plan.md:34-36`). `Taskfile.yml` sequences provisioning correctly, but the race fallback remains.

- **Fix:** disable broker auto-create so misordering fails loud instead of silently degrading.

Tier 3 — worth knowing, fix opportunistically

- **Uncached embedding calls** on every memory read *and* write (`memory_service/service.py:107,140`) — one HTTP round-trip each, never batched or cached. An LRU keyed on `(model, query)` plus a batched store endpoint reclaims fixed latency per tick.
- **Write-on-read:** every successful memory search commits an `UPDATE … access_count+1` (`service.py:172-178`) — WAL traffic on the read path; make it fire-and-forget or sample it.
- **Percept fan-out is O(villagers) serial Redis round-trips** per civic/chat event, on the single consumer task that also gates reactive wake-ups (`percepts.py:257-282`). Collapse each fan-out into one pipeline `execute()` .
- **Ungated 1 s snapshot loop per bot:** full `buildSnapshot` + `JSON.stringify` + Redis `SET` every second per bot even when nothing changed (`BotSession.ts:325-337`) — 20 writes/sec at idle. Chat handling is O(N^2) under fleet-wide chatter (`chatRouter.ts:41-75`).
- **Dashboard N+1:** the relationship graph fires 1 + N proxied requests per load — 21 today, 101 at 100 villagers (`RelationshipGraph.tsx:39-51`). An aggregate-edges endpoint is the down payment on the twice-deferred BFF.
- **Metrics cardinality landmine:** `civ_player_inventory_items` / `civ_materials_collected_total` label by `player × item` (`metrics.ts:81-93`) — the repo's only entity-level labels; usernames × item vocabulary multiply into thousands of series. Drop `player` or cap tracked items.
- **humanInventory polling:** up to 164 sequential RCON round-trips per human per 15 s poll, all through the single-flight RCON socket (`humanInventory.ts:36-89`). Only re-scan when the first pass differs from the last accepted scan.
- **Unbounded growth, no retention:** the ledger table (partitioning deferred, append-only by trigger), memory streams, and `processed_commands` all grow forever. Range-partition the ledger by `occurred_at` before volume climbs.
- **Reconnect herd:** all bots' first reconnect fires in a ~1 s window after a server bounce (`BotSession.ts:298-300` — jitter applies only after the first doubling); combined with Paper's per-IP

`connection-throttle` this is why post-restart recovery needs `connection-throttle: -1`.

- **Redpanda itself runs single-core / 1 GB** (`--smp=1 --memory=1G`, dev-container mode) — fine for 20 villagers, but it is the transport ceiling for any load test; two flags to raise.

What is already good (don't re-fix)

- Villager ticks: one async task each, staggered starts, no global lock, no sync-in-async calls; reactive wake-ups are cooldown- and rate-capped; civic events deliberately do **not** trigger reactive ticks (GPU-stampede guard).
- Ledger reads: true keyset pagination with matching indexes, no `COUNT(*)` in hot paths, GIN index for search; SSE is push-based with bounded, timing-out emitters — not DB polling.
- minecraft-service: 6-way concurrent partition consumption (the only consumer that parallelizes to its partition count), watchdog correctly abandons hung promises, PR #34's `physicsSimCache` / reflex movements / scan gates verified in place.
- Idempotency everywhere it matters: ledger UUIDv7 PK with `ON CONFLICT DO NOTHING`, `votes_one_per_voter`, `processed_commands`, command dedupe + staleness guards.
- LLM budget breakers are per-service with the reflection hourly cap as real backpressure. (Operational note stands: the 2 M default budget trips in ~30 min on Ollama and the fake fallback pollutes narrative state — make the Ollama-sized budget a provider-conditional default rather than a comment.)

Top five actions by leverage

1. **event-service:** listener concurrency + batch inserts + async SSE fan-out — unblocks the system's true serialization point.
2. **commands.minecraft:** raise partitions toward bot count and clamp `timeoutMs` in the executor — kills cross-bot freezes during long actions.
3. **memory-service:** per-query `hnsw.ef_search`, reflection-scan index, and a decay job — stops the slow rot on long-running worlds.
4. **agent-service:** semaphore around LLM calls + producer batching + batched relationship upserts.
5. **infrastructure:** tuned `postgresql.conf`, `mem_limit` on the big three containers, and disable topic auto-create — cheap insurance against the nastiest latent failures.